

STRING MATCHING ALGORITHM

MODULE-3

Knuth-Morris-Pratt Algorithm

- Knuth, Morris and Pratt discovered first linear time string-matching algorithm by analysis of the naive algorithm.
- It keeps the information that naive approach wasted gathered during the scan of the text.
- By avoiding this waste of information, it achieves a running time of **$O(m + n)$** .
- The implementation of Knuth-Morris-Pratt algorithm is efficient because it *minimizes the total number of comparisons* of the pattern against the input string.
- A matching time of $O(n)$ is achieved by avoiding comparison with an element of 'S' that have previously been involved in comparison with some element of the pattern 'p' to be matched. i.e., *backtracking on the string 'S' never occurs*.

Text string

1	2	3	4	5	6	7
a	b	c	a	b	c	d

Pattern

1	2	3	4
a	b	c	d

Components:

- 1. The Prefix Function (Π):** The Prefix Function, Π for a pattern encapsulates knowledge about how the pattern matches against the shift of itself. This information can be used to avoid a useless shift of the pattern 'p.' In other words, this enables *avoiding backtracking of the string 'S.'*
- 2. The KMP Matcher:** With string 'S,' pattern 'p' and prefix function ' Π ' as inputs, find the occurrence of 'p' in 'S' and returns the number of shifts of 'p' after which occurrences are found.

The Prefix Function (Π)

COMPUTE- PREFIX- FUNCTION (P)

1. $m \leftarrow \text{length } [P]$ //'p' pattern to be matched
2. $\Pi [1] \leftarrow 0$
3. $k \leftarrow 0$
4. for $q \leftarrow 2$ to m
5. while $k > 0$ and $P [k + 1] \neq P [q]$
6. $k \leftarrow \Pi [k]$
7. if $P [k + 1] = P [q]$
8. then $k \leftarrow k + 1$
9. $\Pi [q] \leftarrow k$
10. Return Π

Let us consider an example of how to compute u for the pattern 'p'.

Pattern: a b a b a c a

Initially: $m = \text{length}[p] = 7$

$\Pi[1] = 0$

$k=0$

where m , $\Pi[1]$, and k are the length of the pattern, prefix function and initial potential value respectively.

Step 1: $q = 2, k = 0$
 $\pi[2] = 0$

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
π	0	0					

Step 2: $q = 3, k = 0$
 $\pi[3] = 1$

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
π	0	0	1				

Step 3: $q = 4, k = 1$
 $\pi[4] = 2$

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
π	0	0	1	2			

Step 4: $q = 5, k = 2$
 $\pi[5] = 3$

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
π	0	0	1	2	3		

Step 5: $q = 6, k = 3$
 $\pi[6] = 1$

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
π	0	0	1	2	3	0	

Step 6: $q = 7, k = 0$
 $\pi[7] = 1$

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
π	0	0	1	2	3	0	1

After iterating 6 times, the prefix function computations is complete :

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
π	0	0	1	2	3	0	1

The running time of the prefix function is $O(m)$.

Compute Prefix function

1. P=aaaabaacd
2. P=aabaaac
3. P=ABABAABA

Compute Prefix function

1. $P = \text{aaaabaacd} \rightarrow 012301200$
2. $P = \text{aabaaac} \rightarrow 0101220$
3. $P = \text{ABABAABA} \rightarrow 00123123$

The KMP Matcher:

The KMP Matcher with the pattern 'p,' the string 'S' and prefix function ' Π ' as input, finds a match of p in S. Following pseudo code compute the matching component of KMP algorithm:

KMP-MATCHER (T, P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. $\Pi \leftarrow \text{COMPUTE-PREFIX-FUNCTION}(P)$
4. $q \leftarrow 0$ // numbers of characters matched
5. for $i \leftarrow 1$ to n // scan S from left to right
6. while $q > 0$ and $P[q + 1] \neq T[i]$
7. $q \leftarrow \Pi[q]$ // next character does not match
8. if $P[q + 1] = T[i]$
9. $q \leftarrow q + 1$ // next character matches
10. if $q = m$ // is all of p matched?
11. then print "Pattern occurs with shift" $i - m$
12. $q \leftarrow \Pi[q]$ // look for the next match

EXAMPLE

Now let us consider an example so that the algorithm can be clearly understood.

String	b	a	c	b	a	b	a	b	a	b	a	c	a	a	b
--------	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Pattern	a	b	a	b	a	c	a
---------	---	---	---	---	---	---	---

Let us execute the KMP algorithm to find whether 'p' occurs in 'S'.

After iterating 6 times, the prefix function computations is complete :

q	1	2	3	4	5	6	7
p	a	b	A	b	a	c	a
π	0	0	1	2	3	0	1

The running time of the prefix function is $O(m)$.

Initially: $n = \text{size of } S = 15;$
 $m = \text{size of } p = 7$

Step 1: $i = 1, q = 0$
comparing $p[1]$ with $S[1]$

String **b** a c b a b a b a b a c a a b

Pattern **a** b a b a c a

$P[1]$ does not match with $S[1]$. 'p' will be shifted one position to the right.

Step 2: $i = 2, q = 0$
comparing $p[1]$ with $S[2]$

String b **a** c b a b a b a b a c a a b

Pattern **a** b a b a c a

Step 3: $i = 3, q = 1$
comparing $p[2]$ with $S[3]$ $p[2]$ does not match with $S[3]$

String b a **c** b a b a b a b a c a a b

Pattern a **b** a b a c a

Backtracking on p, comparing $p[1]$ and $S[3]$

Step 4: $i = 4, q = 0$
comparing $p[1]$ with $S[4]$ $p[1]$ does not match with $S[4]$

String b a c **b** a b a b a b a c a a b

Pattern **a** b a b a c a

Step 5: $i = 5, q = 0$
comparing $p[1]$ with $S[5]$

String b a c b **a** b a b a b a c a a b

Pattern **a** b a b a c a

Step 6: $i = 6, q = 1$
comparing $p[2]$ with $S[6]$ $p[2]$ matches with $S[6]$

String b a c b **a** b a b a b a c a a b

Pattern **a** b a b a c a

Step 7: $i = 7, q = 2$
comparing $p[3]$ with $S[7]$ $p[3]$ matches with $S[7]$

String b a c b **a** **b** a b a b a c a a b

Pattern **a** **b** a b a c a

Step 8: $i = 8, q = 3$
comparing $p[4]$ with $S[8]$ $p[4]$ matches with $S[8]$

String b a c b **a** **b** **a** b a b a c a a b

Pattern **a** **b** **a** b a c a

Step 9: $i = 9, q = 4$
 comparing $p[5]$ with $S[9]$ $p[5]$ matches with $S[9]$

String b a c b a b a b a b a c a a b

Pattern a b a b a c a

Step 10: $i = 10, q = 5$
 comparing $p[6]$ with $S[10]$ $p[6]$ doesn't matches with $S[10]$

String b a c b a b a b a b a c a a b

Pattern a b a b a c a

Backtracking on p , comparing $p[4]$ with $S[10]$ because after mismatch $q = \pi[5] = 3$

Step 11: $i = 11, q = 4$
 comparing $p[5]$ with $S[11]$

String b a c b a b a b a b a c a a b

Pattern a b a b a c a

Step 12: $i = 12, q = 5$
 comparing $p[6]$ with $S[12]$ $p[6]$ matches with $S[12]$

String b a c b a b a b a b a c a a b

Pattern a b a b a c a

Step 13: $i = 13, q = 6$
 comparing $p[7]$ with $S[13]$ $p[7]$ matches with $S[13]$

String b a c b a b a b a b a c a a b

Pattern a b a b a c a

pattern ' p ' has been found to completely occur in string ' S '. The total number of shifts that took place for the match to be found are: $i - m = 13 - 7 = 6$ shifts.

Run-Time analysis

- $O(m)$ - It is to compute the prefix function values.
- $O(n)$ - It is to compare the pattern to the text.
- Total of **$O(n + m)$** run time.

Practice

1.) Main String: "AAAABAAAAABBBAAAAAB"

The pattern: "AAB"

Pratice

1.) Main String: "AAAABAAAAABBBAAAAAB"
The pattern: "AAAB"

Output:

1

7

14